Reconfigurable Universal DDR Memory Controller

Microarchitecture Specification

DDR / DDR2 / DDR3 / DDR4 / DDR5 LPDDR / LPDDR2 / LPDDR3 / LPDDR4 AXI4 Host Interface DFI PHY Interface

1 Overview

The Reconfigurable Memory Controller (RMC) is a universal DDR controller designed to interface with all major generations of DDR and LPDDR SDRAM, spanning DDR through DDR5 and LPDDR through LPDDR4, without changing the RTL. Protocol-specific behaviour such as timing parameters and initialization sequences are loaded from a software-programmable configuration register file at boot time.

The RMC sits between an AXI4 system interconnect and a standard PHY interface. On the host side it accepts AXI4 burst transactions; on the memory side it drives a DFI-compliant PHY that handles the actual IO.

2 System Architecture

Figure 1 shows where the RMC sits in the full memory subsystem. One or more AXI masters connect through a system interconnect to the MC over AXI or CHI. The MC is the sole block responsible for all DRAM protocol logic, translating host transactions into correctly-timed DDR command sequences and returning read data in order.

The MC drives a universal PHY over a standard PHY interface. The PHY handles IO calibration, timing alignment, and differential signaling, and fans out to whichever DDR or LPDDR generation is physically present on the board. The scope of this document is the MC block only.

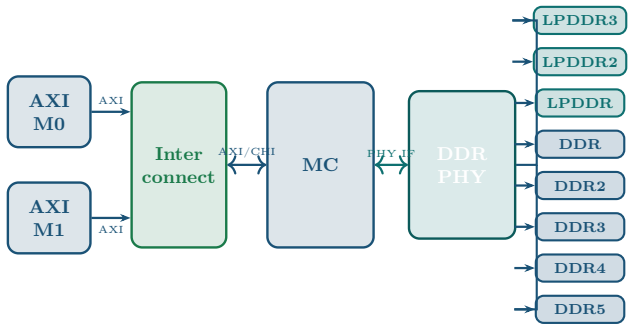

Figure 1: System-level block diagram. The RMC (MC block) is the scope of this document.

3 Client Interface (CIF)

The Client Interface (CIF) terminates the AXI4 subordinate port and converts burst transactions into a stream of normalized read and write commands. It has no knowledge of DRAM timing, bank state, or refresh.

3.1 Write Path

An incoming AXI write burst is first split by the **Burst Splitter** if it crosses a row or bank boundary. The **Address Translator** maps the AXI byte address to a {rank, bank, row, column} tuple using the mapping scheme in the register file. The descriptor and its data are queued in the **Write Command Pool**.

3.2 Read Path

The read path mirrors the write path. Descriptors from the **Read Command Pool** carry a transaction tag that the DRAM domain attaches to returning data. Completed data passes through **Merge Logic**, which reassembles split bursts, and into the **Reorder Buffer**, which retires responses in AXI-ID order back to the master.

4 MC Core

The MC Core owns the clock domain crossing. The **Async FIFO** bridges the AXI clock (client side) and the controller clock (DRAM side), running at half the DDR data rate. Commands cross as normalized `WR_CMD` or `RD_CMD` packets carrying address, data, byte-enable mask, and transaction tag.

The **Config Registers** hold all runtime-programmable parameters: timing values, address mapping mode, page policy, and protocol selection. The **Init FSM** steps through the protocol-specific power-up sequence on reset, reading its command sequence and inter-command delays from the register file, making the same FSM correct for every supported DDR generation.

5 DRAM Domain

The DRAM Domain accepts commands from the internal interface and is solely responsible for all DRAM protocol correctness.

5.1 Scheduling Engines

Three engines feed the central scheduler. The **Bank State Machines** track each physical bank through Idle, Activating, Active, and Precharging states. **Timing Control** maintains per-bank countdown timers for all JEDEC constraints (t_{RCD}, t_{RAS}, t_{RP}, t_{RFC}, t_{FAW}, t_{WTR}, and others from the register file) and presents an eligibility mask each cycle. The **Maintenance Engine** handles auto-refresh, ZQ calibration, power-down entry/exit, and the post-init ready signal.

5.2 DDR Command Scheduler

The scheduler selects one legal command per cycle. Priority: maintenance first (refresh deadlines preempt everything), then row-hit reads, row-hit writes, row-miss reads, and row-miss writes. Open-page vs. close-page policy is configurable per rank.

5.3 Data Paths and DFI

The **Write Data Path** serializes write data onto the DFI bus with correct write-latency alignment and drives byte-mask signals. The **Read Data Path** captures returning DFI data, associates it with the pending transaction tag, and forwards it back to the Client Interface (CIF). All PHY commands are issued over the **DFI Interface**; the RMC never drives DRAM pins directly.

The complete microarchitecture block diagram is on the following page.

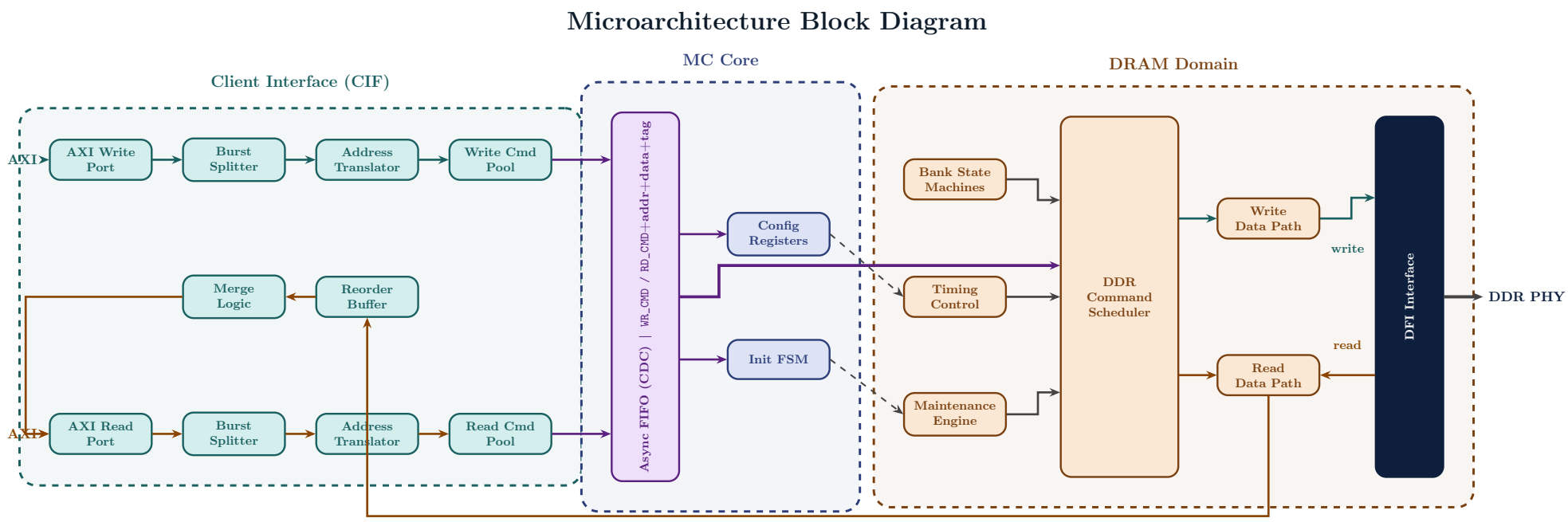

Figure 2: Full microarchitecture block diagram of the RMC. **Teal**: Client Interface (CIF); **Blue**: MC Core; **Amber**: DRAM Domain. The CDC bar is the sole crossing point between the two clock domains.

6 Burst Splitter

The Burst Splitter sits between the AXI port and the Address Translator. Its job is to take an incoming AXI burst and break it down into memory commands the DRAM scheduler can directly consume. This happens in two stages.

Stage 1 checks whether the burst crosses a DRAM row (page) boundary, determined by the address mapping in the config registers. If it does, the burst is split into sub-transactions each confined to a single row. If not, it passes straight through to Stage 2.

Stage 2 checks whether each sub-transaction fits within a single memory command, taking into account the configured burst length and start column address alignment. If not, it gets chopped into multiple memory commands. The output goes into the command pool as `WR_CMD` or `RD_CMD` packets.

Both stages support compile-time enable/disable. When Stage 1 is disabled, incoming AXI bursts must not cross a page boundary. When Stage 2 is disabled, each transaction must already be sized to fit within a single memory command.

AXI Feature Decisions

Feature	Decision	Rationale
Burst type	INCR only	Covers all standard memory traffic. WRAP adds splitter complexity with little benefit. FIXED is not useful for DRAM access.
Data bus width	64-bit	Sufficient for target DDR generations. Can be widened via config registers if needed.
Outstanding IDs	Multiple supported	Needed to hide DRAM latency. The Reorder Buffer handles out-of-order returns and retires responses in AXI-ID order.
Narrow transfers	Not supported	Adds byte-lane steering complexity. All transfers assumed to use the full bus width.
Unaligned transfers	Not supported	High cost, low practical gain. AXI masters are expected to issue aligned bursts.
Exclusive access	Not supported	Requires a reservation table. Out of scope for this design phase.
QoS signals	Not supported	Scheduler complexity not justified at this stage.

Internal Pipeline

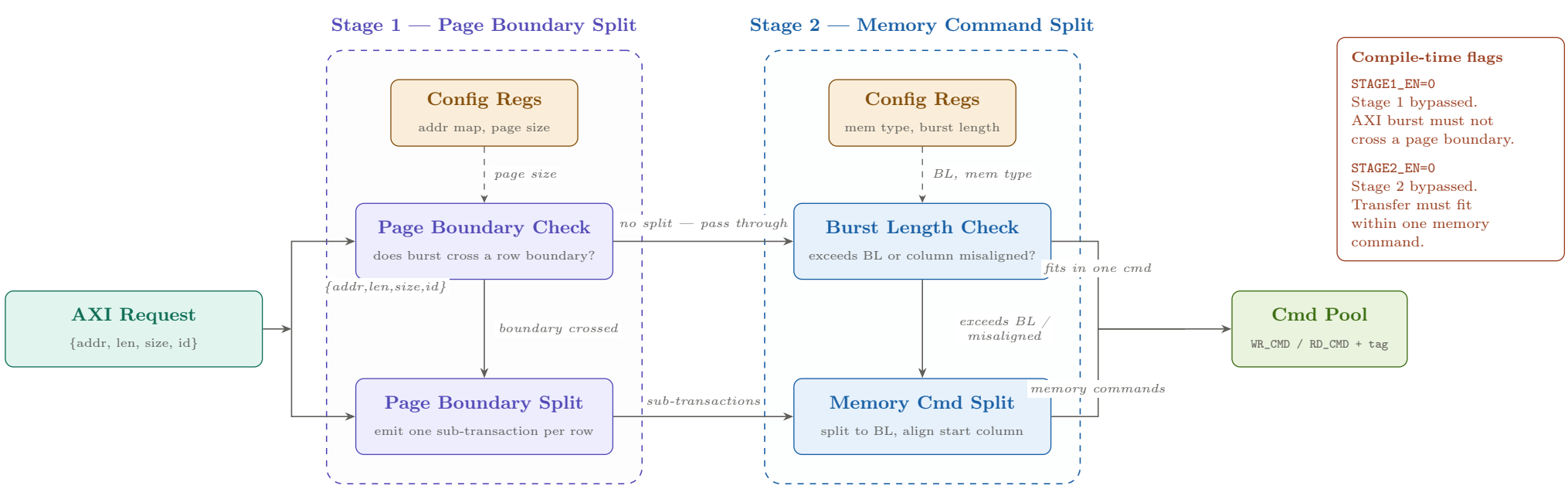

Figure 3: Burst Splitter internal pipeline. Stage 1 splits an AXI request at row boundaries into linear transactions each confined to a single row. Stage 2 divides each into memory commands sized to the configured burst length and aligned to the start column address. Both stages are individually enabled or disabled at compile time.

Address Calculation

All address arithmetic is performed in byte units on the AXI byte address before it is handed to the Address Translator. The two key quantities computed per sub-transaction are the row-boundary address used by Stage 1 and the column-aligned start address used by Stage 2.

Stage 1 — row boundary detection. The active address-mapping mode read from the config registers determines the row-address width R and the column-address width C. The column-address width C defines how many addressable column locations exist per row, so the total number of bytes in one DRAM row is:

$$ROW\_BYTES = 2^{C} \times BUS\_WIDTH\_BYTES$$

where `BUS_WIDTH_BYTES` is 8 for the 64-bit data bus. The byte address of the row boundary immediately above a transaction starting at byte address A is:

$$A_{\text{row\_end}} = \left(\lfloor A / ROW\_BYTES \rfloor + 1\right) \times ROW\_BYTES$$

The burst crosses a row boundary if and only if $A + LEN\_BYTES > A_{\text{row\_end}}$, where $LEN\_BYTES = (\texttt{ARLEN} + 1) \times \texttt{BUS\_WIDTH\_BYTES}$. When a split is required, the first sub-transaction runs from A to $A_{\text{row\_end}}$ minus 1. Subsequent sub-transactions start at successive row boundaries, each of length ROW_BYTES, except the final sub-transaction which covers the remaining bytes.

Stage 2 — burst-length alignment. The configured memory burst length BL sets the atomic transfer granule. For DDR4 BL8 for example, the number of bytes per memory command is:

$$CMD\_BYTES = BL \times BUS\_WIDTH\_BYTES$$

The column-aligned start address of a sub-transaction at byte address A' is:

$$A'_{\text{aligned}} = \lfloor A' / CMD\_BYTES \rfloor \times CMD\_BYTES$$

If A' is not already aligned, meaning A' mod CMD_BYTES is not zero, the first memory command covers the partial beat from A' to the next aligned boundary. The byte-enable mask `BE` is set only for the active lanes in this first command. Each subsequent command is naturally aligned and carries a full-width $\texttt{BE} = \texttt{0xFF}$. The total number of memory commands issued for a sub-transaction of N_{bytes} bytes starting at A' is:

$$N_{\text{cmds}} = \left\lceil \frac{(A' \bmod CMD\_BYTES) + N_{\text{bytes}}}{CMD\_BYTES} \right\rceil$$

Each emitted `RD_CMD`/`WR_CMD` packet carries {`rank, bank, row, col`} derived by the Address Translator from A'_{aligned}, plus the byte-enable mask and the transaction tag.

7 Reorder Buffer (ROB)

The Reorder Buffer restores AXI read-response order after DRAM-domain reordering. It operates entirely within the client clock domain; the Async FIFO CDC boundary is upstream of it.

The DRAM scheduler reorders read commands across AXI IDs to maximise row-hit rate. AXI4 requires that responses to the same AXID are returned in issue order, but places no ordering constraint across different AXIDs. A naive single-queue ROB would stall all AXIDs whenever one AXID's data is delayed, which eliminates the scheduling benefit entirely.

The ROB solves this using a composite tag {AXID, seqnum}. Each AXID has a dedicated counter that increments strictly with every issued read command. This seqnum, combined with the AXID, forms a globally unique tag for every in-flight request. The tag is embedded into the `RD_CMD` by the Tag Insertion block. The tagged command then exits the ROB and crosses the Async FIFO CDC boundary into the DRAM domain.

On the return path, data arrives back from the DRAM domain as {tag, data} pairs. The Tag Lookup and Fill block decodes the tag to recover the AXID and seqnum, writes the data into the correct slot in `queue[AXID][seqnum]`, and sets the slot valid bit. This write is non-blocking, meaning it does not wait for earlier slots of the same AXID to be filled first.

Retirement Logic maintains a per-AXID HEAD pointer, initialised to seqnum 0. Each cycle, if the slot at HEAD is valid, it is retired: the data is forwarded to the AXI Read Port with the correct `RID` and the HEAD pointer advances to the next seqnum. Each AXID retires independently, so a stalled AXID never blocks another.

The following table summarizes the key correctness properties of the ROB design:

Property	Guarantee
Same-AXID ordering	Responses retire in strict seqnum order (HEAD advances only when the next slot is valid).
Cross-AXID isolation	Per-AXID queues and HEAD pointers are independent; no AXID can block another.
Tag uniqueness	$\{\text{AXID}, \text{seqnum}\}$ is globally unique for all in-flight commands because seqnum is per-AXID and strictly increasing.
No Async FIFO inside ROB	The CDC boundary sits outside the ROB. The ROB operates on a single (client) clock, simplifying timing closure.

Figure 4 below shows the complete internal architecture of the ROB.

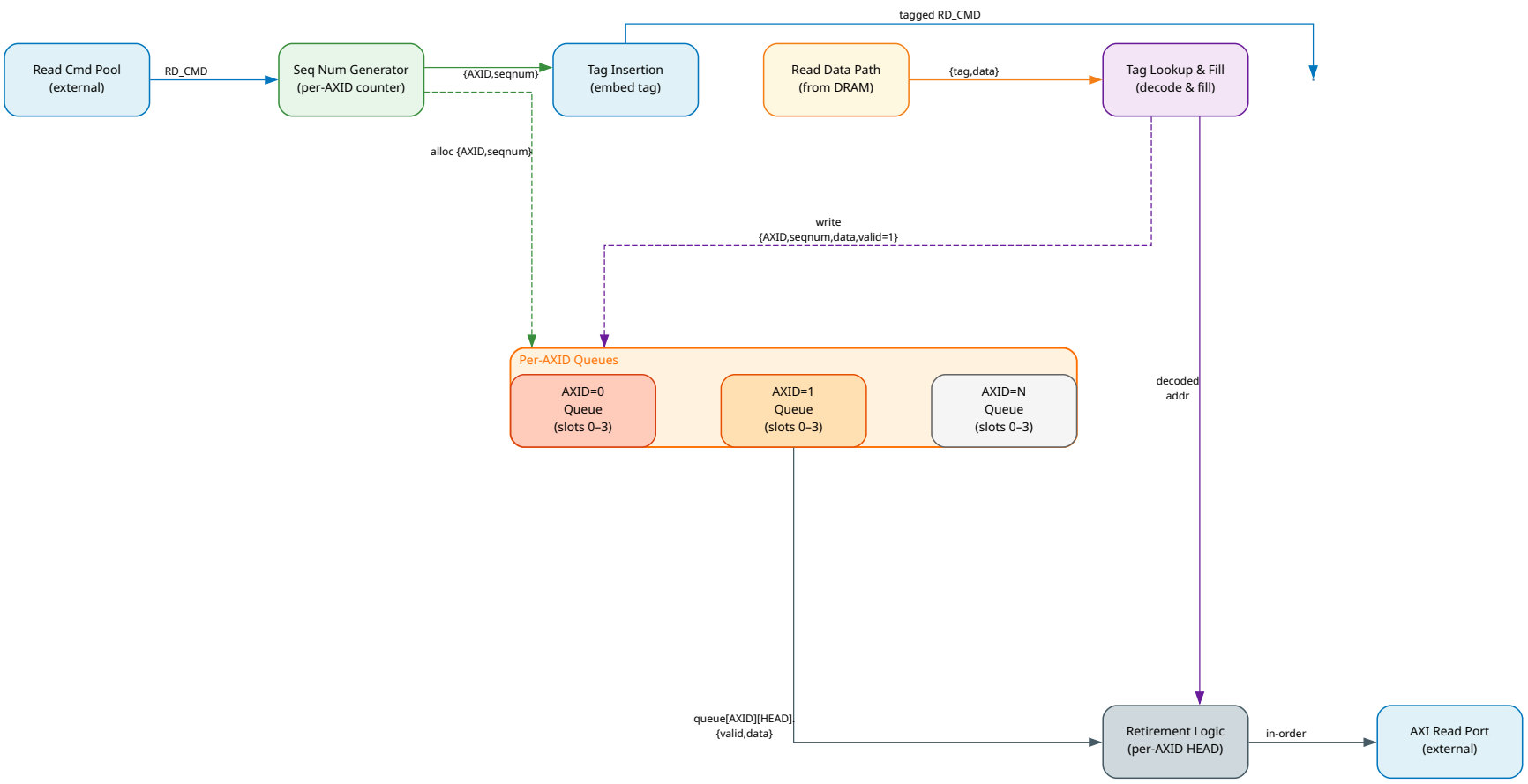

Figure 4: Internal architecture of the Reorder Buffer (ROB). The ROB reconstructs AXI read responses from potentially out-of-order DRAM returns while preserving per-AXID ordering.

8 Address Translator

The Address Translator converts a byte address from the Burst Splitter into the physical DRAM address tuple required by the memory controller. It extracts rank, bank group, bank, row, and column fields from the byte address using a configurable bit-field mapping stored in the Config Registers.

The Bit Field Extractor takes the input byte address and slices it into DRAM address fields based on the configured `addr_map_mode` and the programmed bit widths for each field. The bit boundaries are fully register-programmable, which allows the same RTL to correctly map addresses for DDR3, DDR4, DDR5, and LPDDR4 without any RTL changes. For example, DDR3 has no bank group field, while DDR4 introduces two bank group bits at a different position in the address.

The CMD Builder takes the extracted fields along with the AXI metadata (`AXID, tag, byte-enable mask`) passed through from the input and packs them into a complete `RD_CMD` or `WR_CMD` packet. This packet is then forwarded to the Read or Write Command Pool, where it awaits transmission to the DRAM domain.

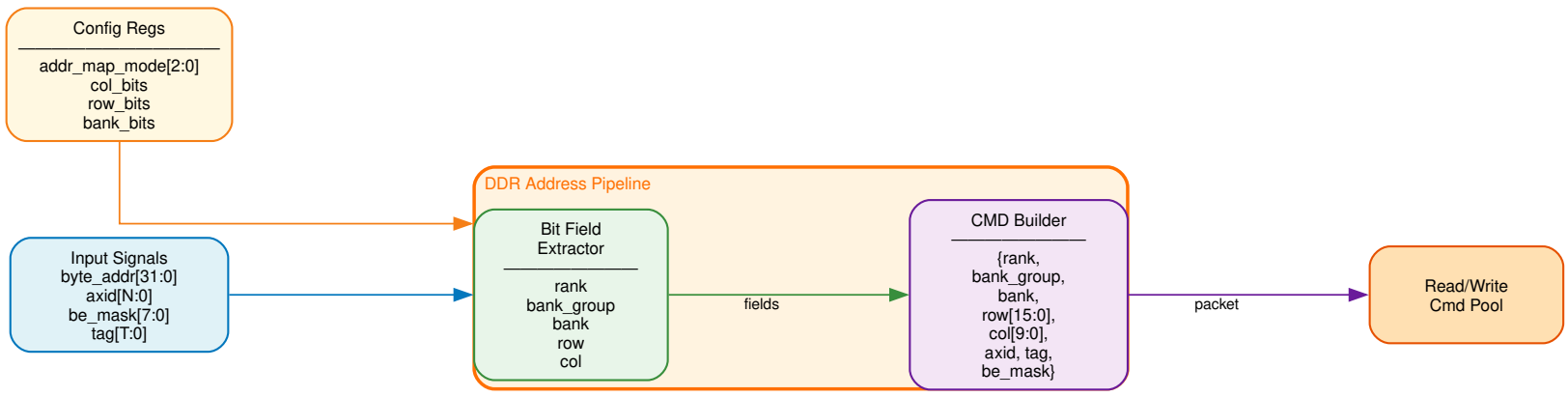

Figure 5: Address Translator pipeline. The Bit Field Extractor decodes the byte address into DRAM fields according to the configured memory map; the CMD Builder packs extracted fields with AXI metadata to form the final command packet.

9 Merge Logic

The Merge Logic reassembles fragments from split DRAM read transactions into complete burst responses before passing them to the Reorder Buffer. When the Burst Splitter divides a single AXI read into multiple sub-commands due to row boundary crossings or burst length constraints, each sub-command is issued independently to the DRAM domain and may return out of order due to DRAM scheduling. Merge Logic is responsible for collecting all fragments belonging to the same original transaction and concatenating them in the correct order.

The Tag Decoder is the first stage. It receives each returning {tag, data} beat from the Read Data Path and extracts the original transaction metadata from the tag field, specifically the `AXID`, original command ID, fragment index, and total fragment count.

The Fragment Buffer stores each arriving data beat at the address `[axid][frag_idx]`. Each slot holds the data word and a valid bit. Fragments can be written in any order since the `frag_idx` field in the tag directly identifies the correct storage location.

The Reassembly Tracker maintains two counters per AXID: `frag_count` tracks how many fragments have arrived, and `expected` holds the total number of fragments for the transaction, latched from `frag_total` when the first fragment arrives. When `frag_count` equals `expected`, the tracker asserts `burst_complete` for that AXID.

The Output Mux reads all stored fragments for the completed transaction in `frag_idx` order from 0 to `frag_total` minus 1, concatenates them into a single data burst, and forwards the result to the ROB. For single-fragment transactions where `frag_total` equals 1, the data bypasses the Fragment Buffer entirely and passes directly to the Output Mux without any buffering.

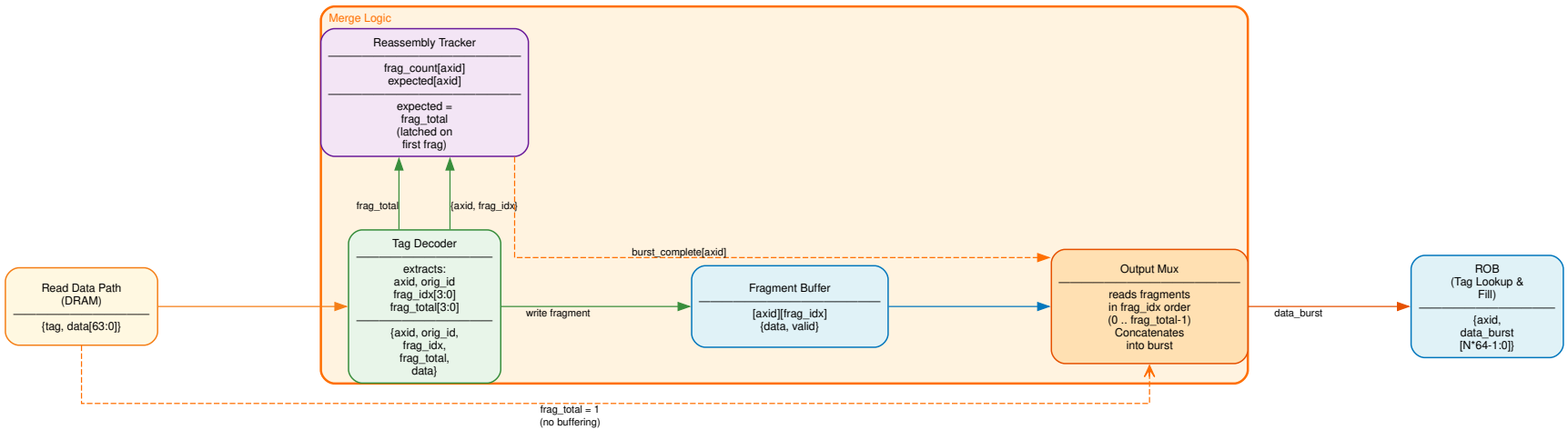

Figure 6: Merge Logic pipeline. The Tag Decoder extracts metadata from returning data; fragments are buffered and tracked for completion; the Output Mux concatenates reassembled bursts in the correct order.